

Project Documentation: Undo/Redo Text Editor

1. Abstract

The Undo/Redo Text Editor is a simplified console-based application designed to demonstrate the practical application of the Stack data structure. The program allows users to perform basic text operations like insertion and deletion while maintaining a history of changes. By utilizing two stacks: a. One for Undo operations and another for Redo operations b. The system ensures efficient state management and recovery of previous text states.

2. Introduction

In modern software applications, the ability to "undo" an accidental action or "redo" a reverted one is a fundamental user requirement. This project implements these features using a Command-Line Interface (CLI) in C++. The core logic relies on the Last-In, First-Out (LIFO) property of stacks to store snapshots of the text string. Every time a destructive or additive change occurs, the current state is saved, allowing the user to navigate through the timeline of their edits seamlessly.

3. Algorithm

The program operates on the following algorithmic principles:

A. Insertion

1. Push the currentText into the undoStack.
2. Append the new text to the currentText.
3. Clear the redoStack (as a new branch of history is created).

B. Deletion (Last Word)

1. Push the currentText into the undoStack.
2. Find the last space in the currentText and truncate the string from that position.
3. Clear the redoStack.

C. Undo

1. Check if the undoStack is empty.
2. If not empty, push the currentText into the redoStack.
3. Set currentText to the top of the undoStack.
4. Pop the top element from the undoStack.

D. Redo

1. Check if the redoStack is empty.
2. If not empty, push the currentText into the undoStack.
3. Set currentText to the top of the redoStack.
4. Pop the top element from the redoStack.

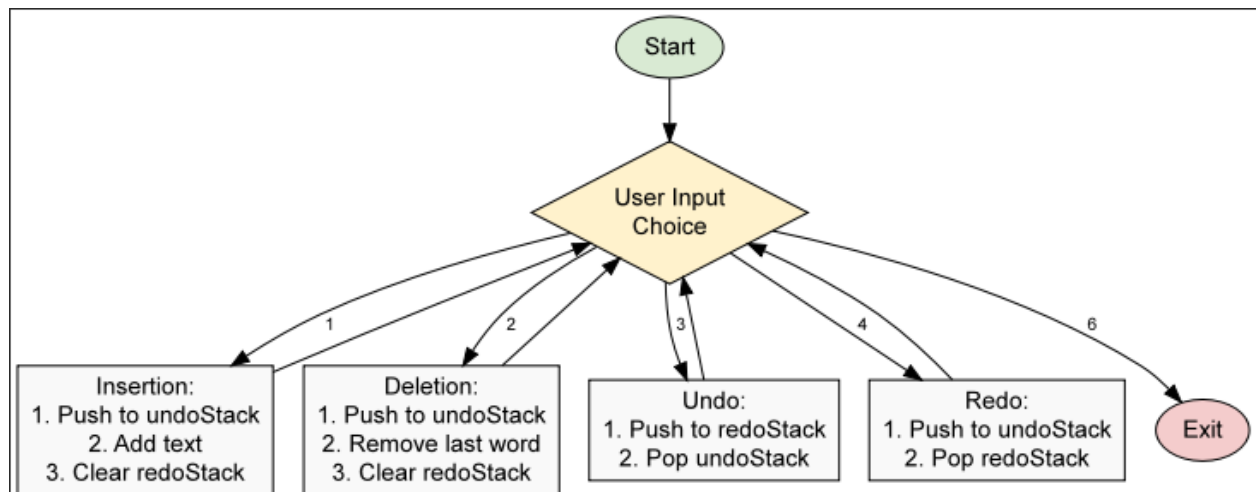
4. Flowchart

Start -> Display User Input Menu

Action -> Update Current State & Save to Stack

Undo/Redo -> Swap states between Stacks

Loop until Exit signal.

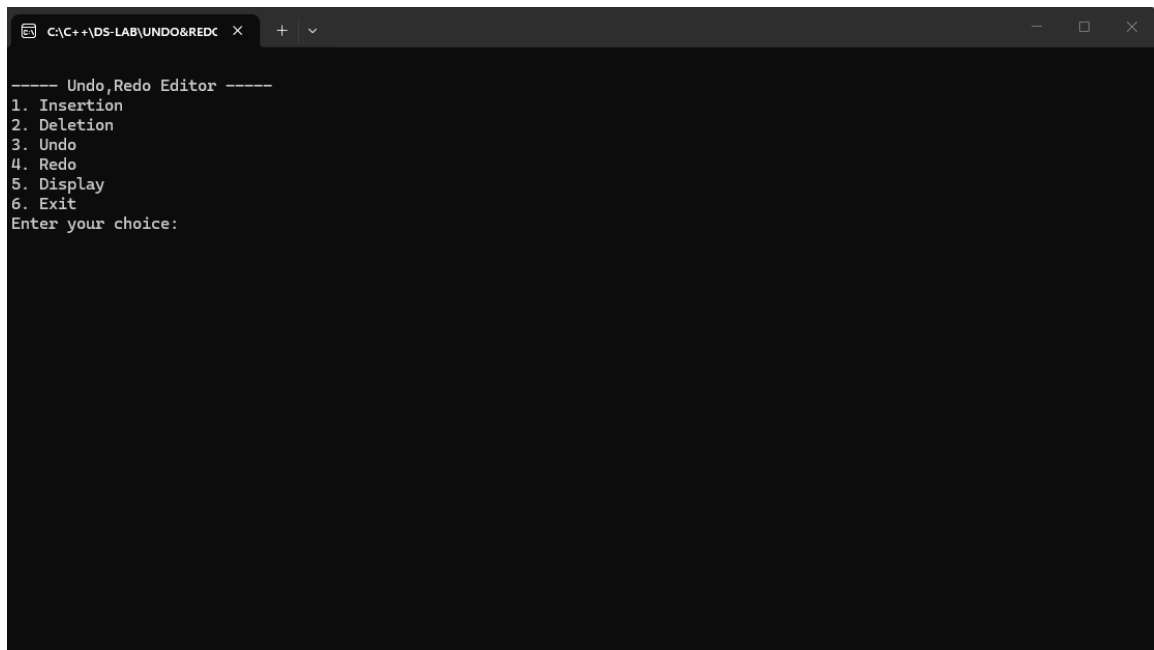


5. Methodology

- The project is implemented in C++ using the Standard Template Library (STL).
 - Data Structure: `std::stack<std::string>` is used for both undo and redo histories.
 - String Handling: `std::string` is used to manage the text content.
 - Control Flow: A `while(true)` loop combined with a switch statement handles the menu-driven user interface.
 - State Management: Each action (Insertion/Deletion) is treated as a "state-changing" event that triggers a push to the undoStack.

6. Result Screenshots

Main Menu



A screenshot of a Windows application window titled "C:\C++\DS-LAB\UNDO&REDO". The window contains a menu titled "----- Undo,Redo Editor -----" with the following options: 1. Insertion, 2. Deletion, 3. Undo, 4. Redo, 5. Display, and 6. Exit. Below the menu, it says "Enter your choice:".

```
----- Undo,Redo Editor -----
1. Insertion
2. Deletion
3. Undo
4. Redo
5. Display
6. Exit
Enter your choice:
```

After Insertion



A screenshot of the application window showing the state after several operations. It displays the menu and the current text "Current Text: RamaKrishna Yelubandi". It shows the user selecting option 5 (Display), then option 3 (Undo), which results in "Undo successful!". It then shows the user selecting option 5 (Display) again, resulting in "Current Text: RamaKrishna". Finally, it shows the user selecting option 6 (Exit), resulting in "Enter your choice:".

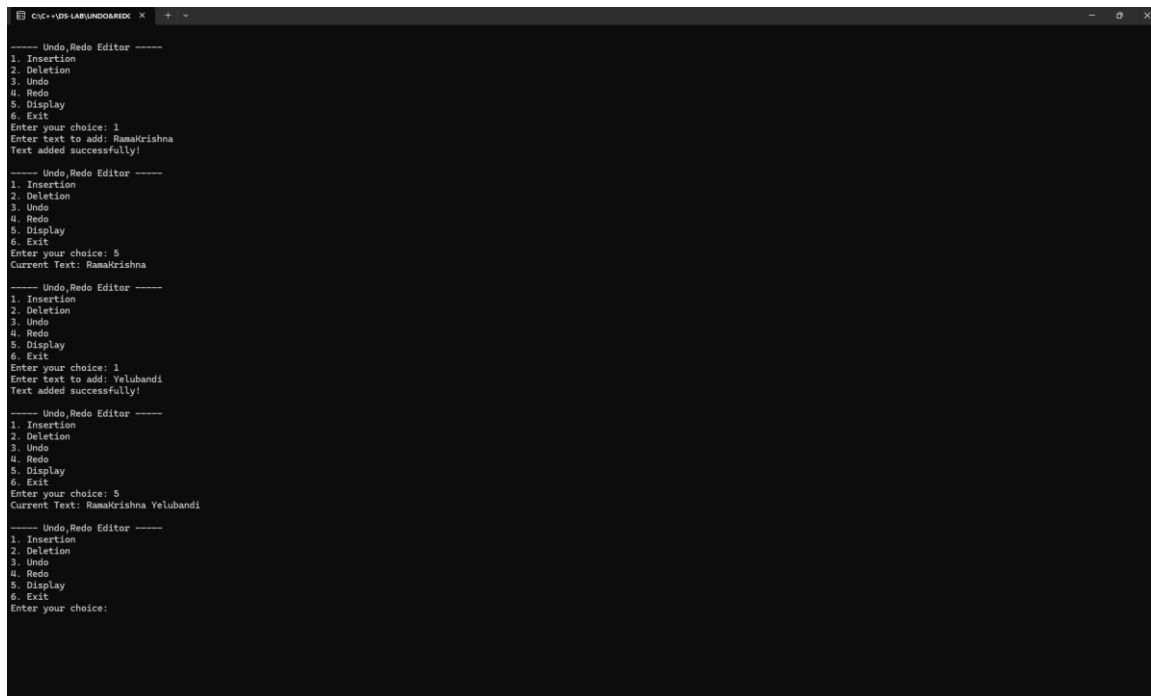
```
----- Undo,Redo Editor -----
1. Insertion
2. Deletion
3. Undo
4. Redo
5. Display
6. Exit
Enter your choice: 5
Current Text: RamaKrishna Yelubandi

----- Undo,Redo Editor -----
1. Insertion
2. Deletion
3. Undo
4. Redo
5. Display
6. Exit
Enter your choice: 3
Undo successful!

----- Undo,Redo Editor -----
1. Insertion
2. Deletion
3. Undo
4. Redo
5. Display
6. Exit
Enter your choice: 5
Current Text: RamaKrishna

----- Undo,Redo Editor -----
1. Insertion
2. Deletion
3. Undo
4. Redo
5. Display
6. Exit
Enter your choice:
```

After Undo operation



```
----- Undo,Redo Editor -----
1. Insertion
2. Deletion
3. Undo
4. Redo
5. Display
6. Exit
Enter your choice: 1
Enter text to add: Ramakrishna
Text added successfully!

----- Undo,Redo Editor -----
1. Insertion
2. Deletion
3. Undo
4. Redo
5. Display
6. Exit
Enter your choice: 5
Current Text: Ramakrishna

----- Undo,Redo Editor -----
1. Insertion
2. Deletion
3. Undo
4. Redo
5. Display
6. Exit
Enter your choice: 1
Enter text to add: Yelubandi
Text added successfully!

----- Undo,Redo Editor -----
1. Insertion
2. Deletion
3. Undo
4. Redo
5. Display
6. Exit
Enter your choice: 5
Current Text: Ramakrishna Yelubandi

----- Undo,Redo Editor -----
1. Insertion
2. Deletion
3. Undo
4. Redo
5. Display
6. Exit
Enter your choice:
```

7. Conclusion

The project successfully demonstrates how stacks can be used to implement complex features like undo and redo logic. The application is robust for basic word-level manipulations and provides a clear understanding of state-based memory management in computer science.

8. Future Work

- Character-level Undo: Currently, it operates on words; future versions could track individual keystrokes.
- File I/O: Adding the ability to save and load text from .txt files.
- GUI Integration: Developing a graphical user interface using frameworks like Qt or SFML.
- Multiple Iterations: Allowing a history limit to optimize memory usage.